

用户要求

我现在还是对你规划不满足：

- ① 请将 scripts、results、build放到相应地码地文件夹中，不要放到码文件外卖呢，否则太混乱；
- ② common文件夹可以放到3个码地文件夹外；
- ③ 公共帧池按照你说的来，但是先要考虑好：后续如果仿真 50000帧，那么目前地帧数不够该怎么办？直观形象地解释你的做法；
- ④ 公共的标准高斯母噪声地做法，我认可；
- ⑤ 在每一个码地文件夹中，比如进行整块编码，那么相应地阶段代码和结果等放到这里面，且以 stage xx命名第几次地实验记录；

若进行分块编码，那么建立一个和整块编码一致地文件夹，并在里面放置相应地阶段代码和结果等放到这里面，且以 stage xx命名第几次地实验记录；

卷积码和LDPC码也是一样地道理；

- ⑥ 还有就是，我后续可能会让codex将当前修改或增加地代码文件git到我的github仓库中，然后我打算使用GPT网页端来检查github仓库中地修改变化，这样我就不用每次都上传代码文件。因此你需要到时候给提示词地时候，让codex将修改或者增加地文件名写到一个md文件中，并标记相应地修改位置，起到地作用，以及该文件在当前工程中地位置；你还有什么更好的方法呢？也可以提出优化方案；

结合之前地对话历史，将所有地修改完善后地规划完整详细地输出出来

完整规划回复

你的这次修改是合理的，而且比我上一版的目录组织更符合你长期维护、人工检查和后续 GitHub 审查的习惯。

核心原则应调整为：

`\boxed{\text{公共输入放 Common；每一种码的代码、脚本、编译产物和结果完全自包含}}`

也就是说：

- Common 可以放在 BCH、CC、LDPC 外部；
- BCH 的 `scripts/build/results` 只能放在 BCH 内；
- CC 的 `scripts/build/results` 只能放在 CC 内；
- LDPC 的 `scripts/build/results` 只能放在 LDPC 内；
- 不同组织方式，例如整块编码、分块编码，也必须各自独立；
- 每一次实验都用 `stageXX` 固定记录，不能把新实验覆盖到旧实验上。

老师原始任务仍要求：BCH 覆盖 200/300 bit 分块与整块方案，卷积码覆盖整块、按时隙分块、硬/软判决和滑窗译码，LDPC 采用短块整块适配；仿真输出 BER、FER、译码时延、编码增益和复杂度，并将交织结果单独汇总。

下面给出修改后的完整工程规划。

一、最终顶层目录结构

你当前项目根目录是：

`C:\Users\V3169\Desktop\Project\SCL_Channel_Coding_Design`

建议最终整理为：

```
SCL_Channel_Coding_Design/
|
|— 任务要求/
|   |— 附件3-信道编码、交织与译码方案及仿真分析.docx
|   |— 附件3-信道编码、交织与译码方案及仿真分析.pdf
|   |— 老师确认事项.md
|   |— 需求变更记录.md
|
|— 初始规划/
|   |— overall_plan.md
|   |— frozen_requirements.md
|   |— open_questions.md
|   |— project_progress.md
|
|— Task/
|   |
|   |— Common/
|   |
|   |— BCH/
|   |
|   |— CC/
|   |
|   |— LDPC/
```

顶层 Task 中只保留四个目录：

- Common
- BCH
- CC
- LDPC

不再在 Task 根目录设置公共的：

- scripts
- build
- results

这些全部进入对应编码目录。

二、Common 公共目录如何设计

建议：

```
Task/
├── Common/
│   ├── frame_pool/
│   │   ├── smoke/
│   │   ├── prescan/
│   │   ├── formal/
│   │   └── metadata/
│   ├── noise_pool/
│   │   ├── smoke/
│   │   ├── validation/
│   │   └── metadata/
│   ├── config/
│   │   ├── global_definition.csv
│   │   ├── stop_rules.csv
│   │   ├── seed_policy.csv
│   │   └── common_format_version.txt
│   ├── formats/
│   │   ├── frame_pool_format.md
│   │   ├── noise_pool_format.md
│   │   ├── result_csv_format.md
│   │   └── checkpoint_format.md
│   ├── scripts/
│   │   ├── generate_frame_pool.py
│   │   ├── extend_frame_pool.py
│   │   ├── generate_noise_pool.py
│   │   ├── inspect_frame_pool.py
│   │   ├── inspect_noise_pool.py
│   │   └── check_common_integrity.py
│   └── docs/
│       ├── common_data_design.md
│       ├── seed_reproducibility.md
│       ├── ebn0_llr_definition.md
│       └── common_change_log.md
```

这里的 Common/scripts 只负责公共数据，不负责 BCH、卷积码或 LDPC 的编译和运行。

三、公共帧池如何支持将来 50000 帧

你担心的是：

现在只生成 100 帧或者 2000 帧，将来 formal 需要 50000 帧时怎么办？

最直观的做法不是每次推翻重建，而是把公共帧池理解成一个可以持续增加页数的“公共试题本”。

1. 形象理解

假设现在创建一本 200 bit 的公共帧册：

```
frame 0
frame 1
frame 2
...
frame 99
```

现在 smoke 只需要 100 帧，因此先写到第 99 帧。

后来 prescan 需要 2000 帧，不删除前面 100 帧，而是继续往后追加：

```
frame 100
frame 101
...
frame 1999
```

以后 formal 需要 50000 帧，再继续追加：

```
frame 2000
frame 2001
...
frame 49999
```

这样原先的 frame 0~1999 完全不变化。

因此：

- smoke 使用前 100 帧；
- prescan 使用前 2000 帧；
- formal 最多使用前 50000 帧；
- 三种码读取的 frame 0 永远是同一组信息；
- 以前已经完成的实验仍然可以精确复现。

核心原则是：

\boxed{\text{只向后追加，不重新生成，不修改旧帧}}

2. 推荐的数据文件形式

不建议直接生成一个巨大 CSV 文件。50000 帧 $\times$ 300 bit 虽然也不算特别大，但 CSV 体积较大、读写慢，而且容易人工误改。

建议 formal 使用分片二进制文件：

```

Common/frame_pool/formal/
├── k200/
│   ├── frames_000000_009999.bin
│   ├── frames_010000_019999.bin
│   ├── frames_020000_029999.bin
│   ├── frames_030000_039999.bin
│   └── frames_040000_049999.bin
└── k300/
    ├── frames_000000_009999.bin
    ├── frames_010000_019999.bin
    ├── frames_020000_029999.bin
    ├── frames_030000_039999.bin
    └── frames_040000_049999.bin

```

每个文件保存 10000 帧。

这样：

- 需要 100 帧时，只读第一个文件的前 100 帧；
- 需要 5000 帧时，只读第一个文件的前 5000 帧；
- 需要 50000 帧时，读完 5 个文件；
- 后续若需要 100000 帧，只需继续增加新分片。

3. smoke 阶段可以使用 CSV

为了人工查看方便：

```

Common/frame_pool/smoke/
├── k200_smoke_100.csv
└── k300_smoke_100.csv

```

CSV 适合人工检查：

```
frameIndex,b0,b1,b2,...,b199
```

但 formal 不建议全部保存成 CSV。

4. 帧池必须配套元数据

例如：

```

Common/frame_pool/metadata/
├── k200_manifest.json
└── k300_manifest.json

```

内容应包括：

```

{
  "payloadLength": 200,
  "masterSeed": 20260715,
  "generator": "mt19937",
  "totalFrames": 50000,
  "shardSize": 10000,
  "formatVersion": "1.0",
  "immutablePrefixFrames": 50000
}

```

其中最重要的是：

```
totalFrames
masterSeed
generator
formatVersion
```

这样后续任何人都知道帧池是怎么生成的。

5. 帧池扩展规则

`extend_frame_pool.py` 应做到：

1. 读取现有 manifest ；
2. 确认已有帧数 ；
3. 恢复或跳转随机数状态 ；
4. 从已有帧末尾继续生成 ；
5. 不允许覆盖旧文件 ；
6. 生成新 shard ；
7. 更新 manifest ；
8. 计算新 shard 的 SHA256 ；
9. 重新检查旧 shard 哈希是否未变化。

例如：

```
python Common\scripts\extend_frame_pool.py`
--payload-length 300`
--target-frames 50000
```

如果现在已有 2000 帧，程序只生成：

$$50000-2000=48000$$

个新帧。

6. 更稳妥的生成方式

为了避免“跳过随机数状态”带来的复杂性，推荐让每一帧都由 frame index 派生独立 seed：

$$\text{frameSeed} = f(\text{masterSeed}, K, \text{frameIndex})$$

例如：

```
frameSeed =
    masterSeed
    + payloadLength * 1000003u
    + frameIndex * 10007u;
```

然后每一帧独立初始化随机生成器。

这样有一个很大的优点：

不需要依赖前 49999 帧的随机数状态，也能单独重新生成第 50000 帧。

例如想复现 frame 32768, 只要知道：

- masterSeed；
- payloadLength；
- frameIndex。

即可重新生成。

四、公共标准高斯母噪声池

你的认可方向是正确的。

由于 BCH、CC、LDPC 码长不同，不应直接共享最终噪声向量，而应共享：

$$z_i \sim \mathcal{N}(0,1)$$

然后每种码根据自己的有效码率计算：

$$\sigma^2 = \frac{1}{2R_{\mathrm{eff}}} 10^{\{E_b/N_0/10\}}$$

最后：

$$y_i = x_i + \sigma z_i$$

目录建议

```
Common/noise_pool/  
├── smoke/  
│   ├── gaussian_seed20260715_frames100_length1024.csv  
│   └── metadata.json  
├── validation/  
│   ├── gaussian_fixed_frames.bin  
│   └── metadata.json  
└── metadata/  
    └── noise_generation_policy.md
```

formal 是否预先存 50000 帧噪声

不建议全部存盘。

50000 帧 × 每帧最多约 1000 个 double：

$$50000 \times 1000 \times 8 \approx 400 \text{ MB}$$

每个 SNR 若都保存一份，会迅速膨胀。

正式实验建议通过：

```
masterNoiseSeed  
caseId  
snrIndex  
frameIndex
```

现场确定性生成标准高斯噪声。

只有：

- MATLAB 对比；
- smoke；
- 固定 trace；
- bug 复现帧；

才把噪声写入文件。

五、BCH 文件夹最终结构

BCH 必须自包含：

```
Task/BCH/  
|  
├── block/  
|  
├── segmented/  
|  
├── shared/  
|  
└── README.md
```

这里建议用：

- block：整块 BCH；
- segmented：BCH(15,11,1) 分块；
- shared：仅供 BCH 两种方案共享的 BCH 内部公共模块。

不要把 BCH 的公共实现放到顶层 Common，因为它不能被 CC、LDPC 复用。

1. BCH 分块编码目录

```
BCH/  
├── segmented/  
|  
├── stages/  
|   ├── stage01_bch15_encoder/  
|   ├── stage02_bch15_decoder/  
|   ├── stage03_frame_adapter/  
|   ├── stage04_matlab_validation/  
|   ├── stage05_awgn_smoke/  
|   ├── stage06_awgn_prescan/  
|   └── stage07_awgn_formal/  
|  
├── current/  
|   ├── include/  
|   ├── src/  
|   └── tests/  
|  
├── scripts/  
|   ├── build.py  
|   ├── run_tests.py  
|   ├── run_smoke.py  
|   └── run_prescan.py
```



```

|   ├── run_formal.py
|   └── compare_matlab.py
|
|   ├── build/
|   |   ├── debug/
|   |   └── release/
|   |
|   ├── results/
|   |   ├── stage01/
|   |   ├── stage02/
|   |   ├── stage03/
|   |   ├── stage04/
|   |   ├── stage05/
|   |   ├── stage06/
|   |   └── stage07/
|   |
|   ├── matlab/
|   |   ├── bch15_reference.m
|   |   ├── bch_segmented_reference.m
|   |   └── compare_cpp_matlab.m
|   |
|   ├── config/
|   ├── docs/
|   └── CHANGELOG.md

```

2. BCH 整块编码目录

```

BCH/
└── block/
    |
    ├── stages/
    |   ├── stage08_shortened_bch_definition/
    |   ├── stage09_gf_arithmetic/
    |   ├── stage10_bm_decoder/
    |   ├── stage11_matlab_validation/
    |   ├── stage12_awgn_smoke/
    |   ├── stage13_awgn_prescan/
    |   └── stage14_awgn_formal/
    |
    ├── current/
    |   ├── include/
    |   ├── src/
    |   └── tests/
    |
    ├── scripts/
    ├── build/
    ├── results/
    ├── matlab/
    ├── config/
    ├── docs/
    └── CHANGELOG.md

```

整块和分块目录结构保持一致，这一点完全符合你的要求。

六、stageXX 应该如何使用

你希望每次实验记录按 stageXX 保存，这非常正确。

但要避免把完整代码复制很多份，导致难以判断哪个是最新版本。

我建议采用：

```
current/  
stages/  
results/
```

三层结构。

1. current

保存当前正式代码。

例如：

```
BCH/segmented/current/src/bch15_encoder.cpp
```

2. stages/stageXX_*

保存该阶段的任务记录和冻结快照，不一定把整个工程全部复制。

例如：

```
stage05_awgn_smoke/  
├── stage_plan.md  
├── changed_files.md  
├── validation_report.md  
├── frozen_config.csv  
├── git_commit.txt  
└── snapshot/
```

snapshot 中只保存本阶段新建或修改的关键代码文件，或者保存对应 Git commit，不必复制所有未修改文件。

3. results/stageXX

保存该阶段运行结果：

```
results/stage05/  
├── summary.csv  
├── frame_detail.csv  
├── ber_fer.png  
├── run_log.txt  
├── config_used.csv  
├── gate_report.md  
└── completion.flag
```

这样不会出现：

- 代码在 stage05；
- 结果却写到公共 formal；
- 配置不知道来自哪个版本；

的问题。

七、卷积码目录最终结构

建议：

```

Task/CC/
|
├─ block/
├─ segmented/
├─ shared/
└─ README.md

```

其中：

- block：300 bit 整块、零尾卷积编码；
- segmented：按时隙分块、连续编码、滑窗 Viterbi；
- shared：trellis、puncture pattern、基础 Viterbi 数据结构。

block

```

CC/block/
├─ current/
│   └─ include/
│       └─ src/
│           └─ tests/
│
├─ stages/
│   └─ stage15_trellis/
│       └─ stage16_rate12_encoder/
│           └─ stage17_hard_viterbi/
│               └─ stage18_soft_viterbi/
│                   └─ stage19_matlab_validation/
│                       └─ stage20_puncture_23/
│                           └─ stage21_puncture_34/
│                               └─ stage22_awgn_smoke/
│                                   └─ stage23_awgn_prescan/
│                                       └─ stage24_awgn_formal/
│
├─ scripts/
├─ build/
├─ results/
├─ matlab/
├─ config/
├─ docs/
└─ CHANGELOG.md

```

segmented

```

CC/segmented/
├─ current/
├─ stages/
│   └─ stage25_continuous_encoder/
│       └─ stage26_sliding_window_viterbi/
│           └─ stage27_boundary_validation/
│               └─ stage28_awgn_smoke/
│                   └─ stage29_awgn_prescan/
│                       └─ stage30_awgn_formal/
│
├─ scripts/
├─ build/
├─ results/
├─ matlab/
├─ config/
├─ docs/
└─ CHANGELOG.md

```

老师原文件要求卷积码既支持整块，又支持按时隙分块和滑窗译码，因此这两个目录最终都要完成。

八、LDPC 目录最终结构

老师确认 LDPC 使用：

- BG2；
- Direct 基图切割；
- 动态提升因子；
- Layered 译码；
- 输入 200 或 300 bit。

LDPC 按老师要求不进行分块，因此它不需要和 BCH/CC 一样设置真正的 segmented 业务目录。

但你希望各类实现组织一致，可以设置：

```
Task/LDPC/  
|  
├─ block/  
├─ shared/  
└─ README.md
```

block 是唯一正式链路。

```
LDPC/block/  
├─ current/  
|   ├── include/  
|   ├── src/  
|   └─ tests/  
├─ stages/  
|   ├── stage31_bg2_table/  
|   ├── stage32_zc_selector/  
|   ├── stage33_direct_matrix_builder/  
|   ├── stage34_hp_rank/  
|   ├── stage35_direct_encoder/  
|   ├── stage36_layered_spa/  
|   ├── stage37_matlab_reference/  
|   ├── stage38_noiseless_validation/  
|   ├── stage39_awgn_smoke/  
|   ├── stage40_awgn_prescan/  
|   └─ stage41_awgn_formal/  
├─ scripts/  
├─ build/  
├─ results/  
├─ matlab/  
├─ config/  
├─ docs/  
└─ CHANGELOG.md
```

原文件中虽然提出 480、576 和不超过 640 bit，但老师已经允许输出码长根据合法提升因子和切割结构动态确定，因此最终报告应记录实际 $\backslash(Z_c)$ 、 $\backslash(n_b)$ 、 $\backslash(N)$ 和有效码率。原始任务要求 LDPC 不分块、不配置交织，这一点继续保留。

九、stage 编号是全局连续还是每个目录重新从 01 开始

这里有两种方案。

方案 A：全项目连续编号

例如：

```
BCH segmented: stage01-stage07
BCH block: stage08-stage14
CC block: stage15-stage24
CC segmented: stage25-stage30
LDPC block: stage31-stage41
```

优点：

- 项目时间线清晰；
- 不会有多个 stage05；
- GitHub 中搜索容易；
- 和你以前 Stage20、Stage21 的习惯一致。

缺点：

- 单个编码文件夹内编号不是从 01 开始。

方案 B：每个方案独立编号

例如：

```
BCH/segmented/stage01
BCH/block/stage01
CC/block/stage01
LDPC/block/stage01
```

优点：

- 每个子项目独立；
- 目录内编号直观。

缺点：

- GitHub 中会出现很多 stage01；
- 写报告时容易混淆。

我更推荐：

`\boxed{\text{全项目 stage 编号连续}}`

并且 stage 名称必须带语义：

```
stage17_cc_hard_viterbi
stage35_ldpc_direct_encoder
```

不要只写：

```
stage17
```

十、每个 stage 的标准内容

每个 stage 目录必须统一包含：

```
stageXX_name/  
├── stage_plan.md  
├── changed_files.md  
├── validation_report.md  
├── frozen_config.csv  
├── commands_used.md  
├── git_commit.txt  
├── known_issues.md  
└── snapshot/
```

其中：

stage_plan.md

记录：

- 本阶段目标；
- 不做什么；
- 输入；
- 输出；
- Gate；
- 停止边界。

changed_files.md

记录新增和修改文件。

validation_report.md

记录测试结果。

frozen_config.csv

保存真实运行参数。

commands_used.md

记录编译和运行命令。

git_commit.txt

记录对应 Git commit hash。

known_issues.md

记录未解决问题。

snapshot

保存本阶段关键修改文件快照，或者只保存 patch。

十一、关于 Codex 修改清单 md 文件

你提出：

Codex 修改或增加代码后，将文件名写入 md，并标记修改位置、作用和工程位置。

这个方法非常有用，但仅写文件名还不够。

建议每个 stage 生成：

changed_files.md

格式如下：

```
# Stage 17 Changed Files

## Summary

本阶段实现卷积码硬判决 Viterbi 译码，不进入软判决或 AWGN formal。

## Added Files

### `Task/CC/block/current/include/viterbi_hard.h`

- 类型：新增
- 作用：声明硬判决 Viterbi 译码入口和结果结构
- 主要接口：
  - `decodeHardViterbi(...)`
- 所属模块：CC / block / decoder

### `Task/CC/block/current/src/viterbi_hard.cpp`

- 类型：新增
- 作用：实现 64 状态 ACS、幸存路径存储和终止状态回溯
- 关键位置：
  - 第 25 – 48 行：分支度量
  - 第 50 – 93 行：ACS 更新
  - 第 101 – 128 行：回溯
- 所属模块：CC / block / decoder

## Modified Files

### `Task/CC/block/current/src/cc_main.cpp`

- 类型：修改
- 作用：增加 `--test-hard-viterbi` 命令入口
- 修改位置：
  - `main()` 参数解析部分
  - 硬判决测试分支
- 原有行为是否改变：否

## Deleted Files

无。

## Generated Files
```

```
### `Task/CC/block/results/stage17/test_summary.csv`
```

- 类型：运行产物
- 作用：保存固定测试帧的译码结果

```
## Validation
```

- noiseless payload mismatch : 0
- single-error test : PASS
- MATLAB output mismatch : 0

这样你以后在 GPT 网页端检查 GitHub 时，可以先看 `changed_files.md`，快速知道：

- 改了什么；
- 为什么改；
- 去哪里看；
- 哪几行最关键；
- 是否影响旧功能。

十二、比 `changed_files.md` 更好的 GitHub 审查方案

建议建立“五件套”。

1. `changed_files.md`

面向你和 GPT 快速阅读。

2. `git diff`

Codex 每次完成后必须生成：

```
git diff --stat  
git diff
```

并把 patch 保存：

```
stageXX_name/changes.patch
```

以后即使 GitHub 不方便查看，也能直接审计 patch。

3. `git_commit.txt`

保存：

```
commit hash  
branch name  
parent commit  
commit message
```

例如：

```
branch: stage17-cc-hard-viterbi  
commit: a1b2c3d4  
parent: e5f6g7h8
```


4. validation_report.md

不能只告诉你代码改了，还要告诉你有没有通过测试。

5. manifest.json

机器可读的修改清单，例如：

```
{
  "stage": "stage17_cc_hard_viterbi",
  "added": [
    "Task/CC/block/current/include/viterbi_hard.h",
    "Task/CC/block/current/src/viterbi_hard.cpp"
  ],
  "modified": [
    "Task/CC/block/current/src/cc_main.cpp"
  ],
  "deleted": [],
  "tests": [
    "test_hard_viterbi_noiseless",
    "test_hard_viterbi_matlab"
  ],
  "gate": "PASS_STAGE17_CC_HARD_VITERBI"
}
```

changed_files.md 给人看， manifest.json 给脚本和 AI 看。

十三、GitHub 分支策略

不要让 Codex 每次直接改主分支。

建议：

```
main
├─ stage01-common-infrastructure
├─ stage02-bch15-encoder
├─ stage03-bch15-decoder
...
```

每个 stage：

1. 从已通过 Gate 的上一个 commit 创建新分支；
2. 只完成本阶段；
3. 运行测试；
4. 生成审计文件；
5. commit；
6. push；
7. 你或 GPT 检查；
8. 通过后 merge；
9. 再进入下一阶段。

例如：

```
git checkout main
git pull
git checkout -b stage02-bch15-encoder
```

完成后：

```
git add .
git commit -m "stage02: implement BCH(15,11) encoder"
git push -u origin stage02-bch15-encoder
```

十四、Codex 每次提交时必须记录哪些内容

建议强制要求 Codex 更新：

```
当前 stage/changed_files.md
当前 stage/validation_report.md
当前 stage/manifest.json
当前 stage/git_commit.txt
对应子项目/CHANGELOG.md
初始规划/project_progress.md
```

其中 `project_progress.md` 只记录全局进度：

```
| Stage | Module | Status | Commit | Gate |
|---|---|---|---|---|
| stage01 | Common infrastructure | PASS | abc123 | PASS_STAGE01 |
| stage02 | BCH(15,11) encoder | PASS | def456 | PASS_STAGE02 |
| stage03 | BCH(15,11) decoder | IN PROGRESS | — | — |
```

这样你打开一个文件，就知道整个项目做到哪里了。

十五、避免 Codex 自动提交错误代码

提示词中应要求：

- Codex 可以修改和测试；
- 只有测试通过后才允许 commit；
- push 是否执行根据你当次指令；
- 不允许自动 merge；
- 不允许强制 push；
- 不允许修改历史 commit；
- 不允许删除旧 stage；
- 不允许覆盖旧结果；
- 不允许进入下一 stage。

最安全的流程是：

```
修改
→ 本地测试
→ 生成审计文件
→ 停止
→ 你确认
→ 再 commit/push
```

而不是让 Codex 改完就直接推送。

十六、三个编码目录中的脚本划分

你要求 scripts 放进各码目录，这样做后建议如下。

BCH segmented

```
BCH/segmented/scripts/  
├─ setup_stage.py  
├─ build_debug.py  
├─ build_release.py  
├─ run_unit_tests.py  
├─ run_matlab_compare.py  
├─ run_smoke.py  
├─ run_prescan.py  
├─ run_formal.py  
├─ plot_results.py  
├─ check_results.py  
└─ create_stage_manifest.py
```

BCH block

同样独立一套。

CC block / segmented

各自独立。

LDPC block

独立一套。

这样脚本不会跨编码调用，便于你定位。

不过可以允许脚本通过相对路径读取：

```
../../Common/
```

中的公共帧池和噪声规范。

十七、每个编码目录中的 build

例如：

```
BCH/segmented/build/  
├─ stage01/  
│ └─ debug/  
│ └─ release/  
├─ stage02/  
│ └─ debug/  
│ └─ release/  
└─ current/
```

但是编译产物通常没有长期保留价值，建议：

- current 保存当前可执行文件；
- stage 下只保存必要的二进制哈希和编译日志；
- Git 不提交 .exe、.obj 等大文件；
- Git 提交 CMake、脚本、日志和结果摘要。

.gitignore 应包括：

```
**/build/**/*.  
obj  
**/build/**/*.  
exe  
**/build/**/*.  
pdb  
**/build/**/*.  
ilk  
**/build/**/*.  
lib  
**/build/**/*.  
exp
```

十八、每个编码目录中的 results

例如：

```
LDPC/block/results/  
├── stage38_noiseless_validation/  
├── stage39_awgn_smoke/  
├── stage40_awgn_prescan/  
└── stage41_awgn_formal/
```

每个结果目录统一：

```
stage39_awgn_smoke/  
├── config_used.csv  
├── summary.csv  
├── frame_detail_sample.csv  
├── runtime_summary.csv  
├── ber.png  
├── fer.png  
├── avg_iterations.png  
├── run_log.txt  
├── gate_report.md  
└── completion.flag
```

formal 大型逐帧数据不要全部提交 GitHub。

建议 GitHub 只保存：

- summary；
- 图片；
- gate report；
- 配置；
- 少量 frame sample；
- 哈希；
- 结果说明。

大型原始结果可以本地保存。

十九、严格对应老师任务的完整 Stage 主线

建议采用全局编号。

公共基础

Stage01：公共工程和目录冻结

- 创建最终目录；
- 检查路径；
- 生成 .gitignore；
- 冻结公共格式。

Stage02：公共帧池和 seed

- 200/300 bit；
- 支持扩展；
- shard；
- manifest；
- SHA256。

Stage03：公共标准高斯噪声

- smoke 文件；
- formal 确定性现场生成策略；
- MATLAB/C++ 一致性。

BCH 分块

Stage04：BCH(15,11,1) 编码器

Stage05：syndrome 查表译码器

Stage06：200/300 bit 分块、填充和恢复

Stage07：MATLAB 官方函数对比

Stage08：无噪声与单错验证

Stage09：AWGN smoke

Stage10：AWGN prescan

Stage11：AWGN formal

BCH 整块

Stage12：缩短 BCH 参数冻结

Stage13：有限域运算

Stage14：整块编码器

Stage15：BM/Chien 译码器

Stage16：MATLAB 对比

Stage17：AWGN smoke

Stage18：AWGN prescan

Stage19：AWGN formal

Stage20：BCH 分块与整块综合对比

卷积码整块

Stage21：trellis

Stage22：1/2 零尾编码器

Stage23：硬判决 Viterbi

Stage24：软判决 Viterbi

Stage25：MATLAB 对比

Stage26：2/3 打孔

Stage27：3/4 打孔

Stage28：AWGN smoke

Stage29：AWGN prescan

Stage30：AWGN formal

卷积码分块

Stage31 : 连续状态编码

Stage32 : 滑窗 Viterbi

Stage33 : 边界行为验证

Stage34 : MATLAB 对比

Stage35 : AWGN smoke

Stage36 : AWGN prescan

Stage37 : AWGN formal

Stage38 : 卷积码整块与分块综合对比

LDPC

Stage39 : BG2 表与合法 $\setminus(Z_c\setminus)$

Stage40 : 动态 $\setminus(Z_c\setminus)$ 选择器

Stage41 : BG2 子矩阵切割

Stage42 : $\setminus(H_p\setminus)$ 秩检查和编码求解预计算

Stage43 : Direct 编码器

Stage44 : Layered SPA/BP

Stage45 : MATLAB Direct 独立参考

Stage46 : 无噪声和扰动验证

Stage47 : AWGN smoke

Stage48 : AWGN prescan

Stage49 : AWGN formal

复杂信道和交织

Stage50 : BCH 交织与突发错误

Stage51：卷积码交织与突发错误

Stage52：多径信道

Stage53：频偏与多普勒

Stage54：短时遮挡

Stage55：综合结果与推荐结论

老师原任务中的 S1～S7 最终都可以映射到这些阶段。

二十、关于 smoke 与 formal 帧数

仍然建议：

Smoke

目的：发现程序错误。

```
minFrames = 20
targetFrameErrors = 5
maxFrames = 100
```

Prescan

目的：寻找瀑布区和估算运行时间。

```
minFrames = 300
targetFrameErrors = 30
maxFrames = 2000
```

Formal trial

目的：验证正式停止条件是否合理。

```
minFrames = 5000
targetFrameErrors = 100
maxFrames = 20000
```

Formal

最终根据 trial 冻结，例如：

```
minFrames = 5000
targetFrameErrors = 200
maxFrames = 50000
```

公共帧池则提前支持最多 50000 帧，但程序按停止规则只读取需要的部分。

二十一、你当前应当先做什么

现在第一步仍然不是写 BCH 编码器，而是完成：

Stage01：最终目录结构与工程治理规范冻结。

Stage01 只做：

1. 审查现有目录；
2. 创建 Common、BCH/CC/LDPC 子结构；
3. 创建 block/segmented；
4. 将 scripts/build/results 放回各自方案内部；
5. 建立 Git 规范；
6. 建立 stage 模板；
7. 创建 changed_files 模板；
8. 创建 manifest 模板；
9. 创建项目进度表；
10. 不实现任何编译码算法。

Stage01 Gate：

PASS_STAGE01_PROJECT_STRUCTURE

然后进入：

Stage02：公共帧池与可扩展 seed 机制。

再之后才是：

Stage04：BCH(15,11,1) 编码器。

二十二、最终原则总结

你修改后的工程应遵守以下硬规则：

1. Common 位于三种码之外，只保存真正可复用的数据和规范。
2. scripts/build/results 必须放在各自编码方案目录中。
3. BCH、CC 的整块和分块目录结构完全一致。
4. LDPC 只做整块 Direct BG2 链路，不创建虚假的分块实验。
5. 公共帧池按照输入长度组织，不按照编码类型重复生成。
6. 帧池只能追加，不能重新生成或覆盖旧帧。
7. formal 帧池采用 shard，支持从 100 帧扩展到 50000 帧。
8. 公共噪声采用标准高斯母噪声，各编码使用自己的 σ 。
9. 每次实验使用全局唯一 stageXX_name。
10. 每个 stage 同时保存计划、代码变更、结果、配置、Gate 和 Git commit。
11. Codex 每次必须生成 changed_files.md、manifest.json、validation_report.md 和 patch。

12. 每个 stage 单独分支、单独 commit、先审查后合并。
13. 不允许覆盖历史 stage 和历史结果。
14. 不通过 MATLAB/独立参考验证，不进入 formal。
15. 项目固定顺序为：

\boxed{\text{Common} \rightarrow \text{BCH分块} \rightarrow \text{BCH整块} \rightarrow \text{卷积码整块} \\ \rightarrow \text{卷积码分块} \rightarrow \text{LDPC BG2 Direct} \rightarrow \text{复杂信道和交织}}

这套结构既满足老师文档，又能保证每个编码、每种组织方式、每次实验的代码与结果互不混淆，同时非常适合以后通过 GitHub 让 GPT 审查每一次 Codex 修改。